
BETH Dataset: Real Cybersecurity Data for Anomaly Detection Research

Kate Highnam^{*1} Kai Arulkumaran^{*21} Zachary Hanif^{*3} Nicholas R. Jennings¹

Abstract

We present the BETH cybersecurity dataset for anomaly detection and out-of-distribution analysis. With real “anomalies” collected using a novel tracking system, our dataset contains over eight million data points tracking 23 hosts. Each host has captured benign activity and, at most, a single attack, enabling cleaner behavioural analysis. In addition to being one of the most modern and extensive cybersecurity datasets available, BETH enables the development of anomaly detection algorithms on heterogeneously-structured real-world data, with clear downstream applications. We give details on the data collection, suggestions on pre-processing, and analysis with initial anomaly detection benchmarks on a subset of the data.

1. Introduction

When deploying machine learning (ML) models in the real world, anomalous data points and shifts in the data distribution are inevitable. From a cyber security perspective, these anomalies and dataset shifts are driven by both defensive and adversarial advancement. To withstand the cost of critical system failure, the development of robust models is therefore key to the performance, protection, and longevity of deployed defensive systems.

Current research into the robustness of ML models tends to be based on existing datasets that are widely used within ML for other purposes. For out-of-distribution (OoD) estimation, researchers combine pairs of existing datasets, such as MNIST-FashionMNIST, CIFAR10-CelebA, or CIFAR10-ImageNet32 (Morningstar et al., 2021). Other data for evaluating robustness involve modified datasets; for example, corrupted, perturbed, and shifted images (Hendrycks & Dietterich, 2019; Ostadvala et al., 2019). These are primarily image datasets (LeCun et al., 1998; Cohen et al., 2017; Xiao

et al., 2017; Krizhevsky et al.; Russakovsky et al., 2015; Liu et al., 2015), and sometimes text datasets (Lewis, 1997; Lang, 1995; Hendrycks et al., 2020). As a result, it is unknown how well new methods—in particular, those centred around deep learning (DL)—may generalise beyond these input modalities.

In this paper, we present the BPF-extended tracking honeypot (BETH) dataset¹ as the first cybersecurity dataset for uncertainty and robustness benchmarking. Collected using a novel honeypot tracking system, our dataset has the following properties that make it attractive for the development of robust ML methods: 1) at over eight million data points, this is one of the largest cyber security datasets available; 2) it contains modern host activity and attacks; 3) it is fully labelled; 4) it contains highly structured but heterogeneous features; and 5) each host contains benign activity and at most a single attack, which is ideal for behavioural analysis and other research tasks. In addition to the described dataset, further data is currently being collected and analysed to add alternative attack vectors to the dataset.

There are several existing cyber security datasets used in ML research, including the KDD Cup 1999 Data (Hettich & Bay, 1999), the 1998 DARPA Intrusion Detection Evaluation Dataset (Labs, 1998; Lippmann et al., 2000), the ISCX IDS 2012 dataset (Shiravi et al., 2012), and NSL-KDD (Tavallaee et al., 2009), which primarily removes duplicates from the KDD Cup 1999 Data. Each includes millions of records of realistic activity for enterprise applications, with labels for attacks or benign activity. The KDD1999, NSL-KDD, and ISCX datasets contain network traffic, while the DARPA1998 dataset also includes limited process calls. However, these datasets are at best almost a decade old, and are collected on in-premise servers. In contrast, BETH contains modern host activity and activity collected from cloud services, making it relevant for current real-world deployments. In addition, some datasets include artificial user activity (Shiravi et al., 2012) while BETH contains only real activity. BETH is also one of the few datasets to include both kernel-process and network logs, providing a holistic view of malicious behaviour.

This paper begins with a description of the data collection

^{*}Equal contribution ¹Imperial College London, London, United Kingdom ²ARAYA Inc., Tokyo, Japan ³University of Maryland, College Park, Maryland, USA. Correspondence to: Kate Highnam <k.highnam19@imperial.ac.uk>.

¹<https://www.kaggle.com/katehighnam/beth-dataset>

process and the relevance of the available features. We then perform an analysis of the first set of kernel-level process logs collected, including anomaly detection benchmarks². Our benchmarks include both traditional baselines (Rousseeuw, 1984; Schölkopf et al., 2001; Liu et al., 2008), as well as a state-of-the-art DL-based method (Morningstar et al., 2021). In summary, the isolation forest (iForest) (Liu et al., 2008) achieves the highest AUROC on the initial, labelled subset of our data. We believe the scale and range of attacks available in our full dataset will pose a challenge for all current anomaly detection methods.

2. The BETH Dataset

The BETH dataset currently represents 8,004,918 events collected over 23 honeypots, running for about five non-contiguous hours on a major cloud provider. For benchmarking and discussion, we selected the initial subset of the process logs. This subset was further divided into training, validation, and testing sets with a rough 60/20/20 split based on host, quantity of logs generated, and the activity logged—only the test set includes an attack. Table 1 provides a summary of the dataset, while Table 2 and Table 5 provide a description of the kernel-process and DNS log features, respectively.

In this section, we first detail the log collection methodology, followed by a description of the overall dataset. The final subsection discusses potential research questions that could be investigated using our dataset.

Table 1. General characteristics of the kernel-process logs, including our initial benchmark subset.

DATASET	LENGTH	% OF SUBSET	# OF HOSTS
TRAINING	763,144	66.88%	8
VALIDATION	188,967	16.56%	4
TESTING	188,967	16.56%	1
SUBSET TOTAL	1,141,078	100%	13
TOTAL	8,004,918	-	23

2.1. Collection Methodology

The challenge of crafting a honeypot is two-fold: make it tempting enough to infiltrate, and track activity without being detected. The former is typically done by providing “free” resources to an attacker, i.e., easily accessible computer power. Our implementation currently runs hosts with a single `ssh` vulnerability: any password will be accepted to login. This is protected enough that it could contain valu-

able information or resources within it, but implies that the user simply has a poor password choice. In the future we plan to deploy hosts with other vulnerabilities, with which we hope to observe other attack vectors.

To subtly log activity in real time, each host runs a Docker container (Merkel, 2014) to encapsulate our two-sensor monitoring system utilising the (extended) Berkeley Packet Filter (BPF) (Gregg, 2019). The first sensor is embedded at the kernel level of the Linux OS to listen to and exfiltrate relevant data packets. In particular, this sensor tracks all OS calls to create, clone, and kill processes. The second sensor logs network traffic, specifically DNS queries and responses from all processes on the host machine, including those processes running within the hosted Docker containers. When the desired packet appears, it is parsed out to pre-defined fields and then transmitted to a collection server.

2.2. Dataset Characteristics

The dataset is composed of two sensor logs: **kernel-level process calls** and **network traffic**. As the initial benchmark subset only includes process logs, this section only covers these; a description of the network logs can be found in Appendix B.

Each process call **consists of 14 raw features and 2 labels, described in Table 2**. These largely contain categorical covariates with some containing large integers, necessitating further processing. Thus, for our benchmarking, we converted several fields to binary variables based on field expertise, as described in Appendix A.

Each record was manually labelled suspicious (`sus`) or `evil` to assist analysis. Logs marked suspicious indicate unusual activity or outliers in the data distribution, such as an external `userId` with a `systemd` process³, infrequent daemon process calls (e.g., “`acpid`” or “`accounts-daemon`”), or calls to close processes that we did not observe as being started. `Evil`⁴ indicates a malicious external presence not inherent to the system, such as a `bash` execution call to list the computer’s memory information, remove other users’ `ssh` access, or `un-tar` an added file. Events marked `evil` are considered “out of distribution,” as they are generated from a data distribution not seen during training.

The kernel process logs were divided into a typical 60/20/20 split for training, validation, and testing, based on the observed activity and labels. Our initial training and validation sets each contain logs generated from multiple hosts, containing only activity from the OS and cloud infrastructure management. Activity in each of these hosts was benign

³In the scope of our honeypot any external user traffic is suspicious, but some of these events were initiated by the cloud provider.

⁴We note that presence in this dataset does not constitute a “conviction”, as no real damage was done.

²https://github.com/jinxmirror13/BETH_Dataset_Analysis

Table 2. The description and type of each feature within the kernel-level process logs, tracking every create, clone, and kill process call. Starred features were included in the model baselines and converted as described in Appendix A.

FEATURE	TYPE	DESCRIPTION
TIMESTAMP	FLOAT	SECONDS SINCE SYSTEM BOOT
PROCESSID*	INT	INTEGER LABEL FOR THE PROCESS SPAWNING THIS LOG
THREADID	INT	INTEGER LABEL FOR THE THREAD SPAWNING THIS LOG
PARENTPROCESSID*	INT	PARENT’S INTEGER LABEL FOR THE PROCESS SPAWNING THIS LOG
USERID*	INT	LOGIN INTEGER ID OF USER SPAWNING THIS LOG
MOUNTNAMESPACE*	INT (LONG)	SET MOUNTING RESTRICTIONS THIS PROCESS LOG WORKS WITHIN
PROCESSNAME	STRING	STRING COMMAND EXECUTED
HOSTNAME	STRING	NAME OF HOST SERVER
EVENTID*	INT	ID FOR THE EVENT GENERATING THIS LOG
EVENTNAME	STRING	NAME OF THE EVENT GENERATING THIS LOG
ARGSNUM*	INT	LENGTH OF ARGS
RETURNVALUE*	INT	VALUE RETURNED FROM THIS EVENT LOG (USUALLY 0)
STACKADDRESSES	LIST OF INT	MEMORY VALUES RELEVANT TO THE PROCESS
ARGS	LIST OF DICTIONARIES	LIST OF ARGUMENTS PASSED TO THIS PROCESS
SUS	INT (0 OR 1)	BINARY LABEL AS A SUSPICIOUS EVENT (1 IS SUSPICIOUS, 0 IS NOT)
EVIL	INT (0 OR 1)	BINARY AS A KNOWN MALICIOUS EVENT (0 IS BENIGN, 1 IS NOT)

for the entire duration of their existence, and, as such, we consider these events to be “in-distribution”.

Our initial testing dataset contains all activity on a single exploited host, including its OS and cloud infrastructure management. The first attack we logged is an attempt to setup a botnet. More details on this attack can be found in Appendix C. The full dataset contains other malicious activity performed within our honeypots, including cryptomining and lateral movement (between servers). These various attacks may also be compared to answer alternative research questions with our data, as discussed in Subsection 2.3. As each exploited host only contains a single staged attack, with no artificial noise in the benign activity, BETH is one of the cleanest cyber security datasets available to distinguish malicious from benign.

As an initial investigation of the data, we visualised the (pre-processed) training and testing datasets with uniform manifold approximation and projection (UMAP) (McInnes et al., 2018). UMAP was first fitted to the training set before being used to project the testing set into the same space. As can be seen in Figure 1, the data from both sets forms several large clusters in the centre, surrounded by many smaller clusters, with both benign and malicious activity spread across the entire space. The first image shows significant overlap between the training and testing sets. The second image shows that *evil* events appear in distinct areas. This indicates that unsupervised methods could potentially detect a large portion of the “anomalous” events.

2.3. Research Questions

The BETH dataset could answer other cyber security questions than just OoD analysis. Unlike logs within real de-

ployed systems that contain no labels for malicious events, our BETH dataset contains (recently recorded) real data with labels. One use for this dataset would be to profile the attacker or malware’s behaviour (Chen et al., 2019). For instance, the known *evil* events could form a unique *fingerprint*, a method of uniquely identifying the tactic used by the attacker, to link an attack to its family or appropriate resolution strategy (Brumley et al., 2007). One could also use graph analysis of process relationships to find malicious cliques (Elhadi et al., 2012), or use time series analysis of execution sequences to profile process names (*processName*) on a modern OS. This latter topic is particularly interesting as some attackers rename malicious processes to benign process names to trick systems into running malicious code. The logs would present a benign process name, even if the arguments or events were inconsistent with normal activity.

3. Anomaly Detection Baselines

In this section, we provide anomaly detection benchmarks on our initial subset of logs. We chose both standard anomaly detection baselines (Pedregosa et al., 2011; Yahaya et al., 2021), which includes robust covariance (Rousseeuw, 1984), one-class support vector machine (SVM) (Schölkopf et al., 2001) and iForest (Liu et al., 2008), as well as density of states estimation (DoSE) (Morningstar et al., 2021), which is based on deep generative models. As per Morningstar et al. (2021), we report area under the receiver operating characteristic (AUROC), using an ensemble of 5 models for each method.

Robust covariance (Rousseeuw, 1984) fits an “ellipsoid with the smallest volume or with the smallest covariance determi-



Figure 1. UMAP visualisations of the training and testing dataset using the pre-processed features (see Appendix A). Figure (a) shows the overlap between the training and testing dataset; Figure (b) highlights the trails of evil events.

nant” (Peña & Prieto, 2001) around the central data points; the tightness is controlled assuming a given level of contamination with anomalies (which we set to 0.05). The anomalies are then scored using the Mahalanobis distance. Similarly, the one-class SVM fits a hyperplane to discriminate between the support of the in-distribution data and OoD data (Schölkopf et al., 2001). As kernelised SVMs scale with $O(N^2)$, we instead utilised `scikit-learn`’s linear SVM with stochastic gradient descent, after whitening the data. In contrast, the iForest (Liu et al., 2008) tries to characterise anomalous points in the data distribution using an ensemble of “isolation trees”.

Given the scale of the dataset, we also considered DL-based OoD detection methods. In particular, DoSE uses summary statistics (such as the log-likelihood or posterior entropy) calculated over the training set by a trained generative model in order to characterise the “typical set”. In our work we train a variational autoencoder (VAE) (Kingma & Welling, 2013), modelling the observations as a product of categorical distributions, but otherwise use largely the same setup as the original paper (Morningstar et al., 2021)⁵. However, given the size of the training set, we were only able to use DoSE with a linear one-class SVM trained using SGD.

Table 3. OoD AUROC results.

METHOD	AUROC
ROBUST COVARIANCE	0.519
ONE-CLASS SVM	0.605
iFOREST	0.850
VAE + DoSE (SVM)	0.698

⁵Model and training details are given in Appendix D.

As seen in Table 3, the iForest performs best at differentiating sus events from the benign in our testing dataset. We attribute this to the small set of discrete features available and the conspicuous nature of the attack. DL-based models are less competitive on these sets of features, but have the potential to deal with more raw categorical and even text-based features, which we hope to explore in future work. Finally, we note that imbalanced labelling, summarised in Table 4, necessitates further investigation of what each model predicts is benign or not.

4. Conclusions

In this paper, we present our BETH cybersecurity dataset for anomaly detection and OoD analysis. The data was sourced from our novel honeypot tracking system recording both kernel-level process events and DNS network traffic. It contains real-world attacks in the presence of benign modern OS and cloud provider traffic, without the added complexity of noisy artificial user activity. This cleanliness is ideal for OoD analysis, such that each host in the dataset only contains one or two data-generating distributions. We also include baselines for anomaly detection trained on a subset of the BETH dataset: robust covariance, one-class SVM, iForest, and DoSE-SVM (with a VAE).

For future work, we plan to collect and publish more attacks for alternative testing datasets. This will also allow investigations in comparing attacks or perhaps testing in a continual learning setting.

Acknowledgements

We thank Dr. Arinbjörn Kolbeinsson for inspiring the included UMAP visualisations and the reviewers for the positive feedback.

References

- Brumley, D., Caballero, J., Liang, Z., Newsome, J., and Song, D. Towards automatic discovery of deviations in binary implementations with applications to error detection and fingerprint generation. In *USENIX Security Symposium*, pp. 15, 2007.
- Chen, Q., Islam, S. R., Haswell, H., and Bridges, R. A. Automated ransomware behavior analysis: Pattern extraction and early detection. In *International Conference on Science of Cyber Security*, pp. 199–214. Springer, 2019.
- Cohen, G., Afshar, S., Tapson, J., and Van Schaik, A. Emnist: Extending mnist to handwritten letters. In *2017 International Joint Conference on Neural Networks (IJCNN)*, pp. 2921–2926. IEEE, 2017.
- Elhadi, A. A., Maarof, M. A., and Osman, A. H. Malware detection based on hybrid signature behaviour application programming interface call graph. *American Journal of Applied Sciences*, 9(3):283, 2012.
- Gregg, B. *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional, 1st edition, 2019. ISBN 0136554822.
- Hendrycks, D. and Dietterich, T. Benchmarking neural network robustness to common corruptions and perturbations. *Proceedings of the International Conference on Learning Representations*, 2019.
- Hendrycks, D., Liu, X., Wallace, E., Dziedzic, A., Krishnan, R., and Song, D. Pretrained transformers improve out-of-distribution robustness. *arXiv preprint arXiv:2004.06100*, 2020.
- Hettich, S. and Bay, S. The uci kdd archive [<http://kdd.ics.uci.edu>]. irvine, ca: University of california. *Department of Information and Computer Science*, 152, 1999.
- Kingma, D. P. and Welling, M. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- Krizhevsky, A., Nair, V., and Hinton, G. Cifar-10 (canadian institute for advanced research). URL <http://www.cs.toronto.edu/~kriz/cifar.html>.
- Labs, M. L. 1998 darpa intrusion detection evaluation dataset, 1998. URL <https://www.ll.mit.edu/r-d/datasets/1998-darpa-intrusion-detection-evaluation-dataset>.
- Lang, K. Newsweeder: Learning to filter netnews. In *Machine Learning Proceedings 1995*, pp. 331–339. Elsevier, 1995.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Lewis, D. D. Reuters-21578 text categorization collection data set, 1997.
- Lippmann, R. P., Fried, D. J., Graf, I., Haines, J. W., Kendall, K. R., McClung, D., Weber, D., Webster, S. E., Wyszogrod, D., Cunningham, R. K., et al. Evaluating intrusion detection systems: The 1998 darpa off-line intrusion detection evaluation. In *Proceedings DARPA Information Survivability Conference and Exposition. DIS-CEX’00*, volume 2, pp. 12–26. IEEE, 2000.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. Isolation forest. In *2008 eighth ieee international conference on data mining*, pp. 413–422. IEEE, 2008.
- Liu, Z., Luo, P., Wang, X., and Tang, X. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, December 2015.
- Loshchilov, I. and Hutter, F. Fixing weight decay regularization in adam. 2018.
- McInnes, L., Healy, J., and Melville, J. Umap: Uniform manifold approximation and projection for dimension reduction, 2018. URL <http://arxiv.org/abs/1802.03426>. cite arxiv:1802.03426Comment: Reference implementation available at <http://github.com/lmcinnes/umap>.
- Merkel, D. Docker: lightweight linux containers for consistent development and deployment. *Linux journal*, 2014 (239):2, 2014.
- Morningstar, W., Ham, C., Gallagher, A., Lakshminarayanan, B., Alemi, A., and Dillon, J. Density of states estimation for out of distribution detection. In *International Conference on Artificial Intelligence and Statistics*, pp. 3232–3240. PMLR, 2021.
- Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J. V., Lakshminarayanan, B., and Snoek, J. Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. *arXiv preprint arXiv:1906.02530*, 2019.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Peña, D. and Prieto, F. J. Multivariate outlier detection and robust covariance matrix estimation. *Technometrics*, 43(3):286–310, 2001.

Rousseeuw, P. J. Least median of squares regression. *Journal of the American statistical association*, 79(388):871–880, 1984.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015. doi: 10.1007/s11263-015-0816-y.

Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., and Williamson, R. C. Estimating the support of a high-dimensional distribution. *Neural computation*, 13(7):1443–1471, 2001.

Shiravi, A., Shiravi, H., Tavallae, M., and Ghorbani, A. A. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Computers & Security*, 31(3):357–374, 2012. ISSN 0167-4048. doi: <https://doi.org/10.1016/j.cose.2011.12.012>. URL <https://www.sciencedirect.com/science/article/pii/S0167404811001672>.

Tavallae, M., Bagheri, E., Lu, W., and Ghorbani, A. A. A detailed analysis of the kdd cup 99 data set. In *2009 IEEE symposium on computational intelligence for security and defense applications*, pp. 1–6. IEEE, 2009.

Xiao, H., Rasul, K., and Vollgraf, R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms, 2017.

Yahaya, S. W., Lotfi, A., and Mahmud, M. Towards a data-driven adaptive anomaly detection system for human activity. *Pattern Recognition Letters*, 145:200–207, 2021. ISSN 0167-8655. doi: <https://doi.org/10.1016/j.patrec.2021.02.006>. URL <https://www.sciencedirect.com/science/article/pii/S0167865521000611>.

A. Pre-Processing

In this section, we provide more details on the raw features in the dataset, as well as pre-processing suggestions, which we used in our baselines:

timestamp: We left this field out to consider the dataset as a sample from a distribution rather than time series. We recommend using the values as they are or also leave them out, depending on the method chosen.

processId: Process IDs 0, 1, and 2 are meaningful since these are always values used by the OS, but otherwise a random number is assigned to the process upon creation. We recommend replacing `processId` with a binary variable indicating whether or not `processId` is 0, 1, or 2.

threadId: While this value did not appear useful in our analysis, it might suggest how to link process calls if obfuscated in the system. No conversion is recommended at this time.

parentProcessId: Same as `processId`, the same mapping to a binary variable should suffice.

userId: The default in Linux systems is to assign OS activity to some number below 1000 (typically 0). As users login, they are assigned IDs starting at 1000, incrementally. This can be altered by a user, but none of the current logs gave evidence an attacker did this. We used a binary variable to indicate `userId < 1000` or `userId ≥ 1000`. Alternatively, one could use an ordinal mapping that buckets all `userId < 1000` at zero and then increment upwards for each new user. Also, no more than four logins were viewed per host in our current datasets.

mountNamespace: This field is somewhat consistent across our hosts and determines the access a certain process has to various mount points. The most common value for this feature is 4026531840 or `0xF0000000`, which is for the `mnt/` directory where all manually mounted points are linked. It is noted that all logs with `userId ≥ 1000` had a `mountNamespace` of 4026531840, while some OS `userId` traffic used different `mountNamespace` values. We converted this feature into a binary mapping for whether or not `mountNamespace = 4026531840`.

processName: This is a string field of variable length (ranging from one to fifteen characters). When manually analysing the data, this was a critical field in conjunction with the `eventName`. For our baselines, we refrained from utilising this, although the model should be given an encoding of this using a hash or ability to learn a useful encoding on its own. It is noted that attackers can easily change the `processName` to override a benign one so their traffic looks regular. This was not observed within the current dataset.

hostName: This field is useful for grouping the dataset into related subsets of data generated from the same honey-pot. The name of the host name does not transfer between the model development subsets described in this paper.

eventId: Linux systems assign an integer corresponding to the `eventName`. We include this field as is for our benchmarks.

eventName: Event names uniquely map to `eventId`, so we drop it from training.

argsNum: This raw feature is included as-is, since, at this time, adequately parsing `args` requires either more sophisticated pre-processing or a more complex ML model.

returnValue: This is also called the exit status and can be used to determine whether a call completed successfully or not. Mappings for this can vary, as this value is decided between the parent and child process. We mapped `returnValue` into three values based on the common usage of the field: -1 when negative (bad errors), 0 when zero (success), and 1 when positive (success and signalling something to the parent process).

stackAddresses: It is difficult to clearly relate this feature during manual analysis and the large values within a variable size list make processing automatically difficult without encoding or learning an extra embedding. Thus this field was dropped from training our baselines.

args: There are many options in this variable list of dictionaries. For simplicity, we refrain from utilising any of these values. However, more features can and should be created for future work.

Finally, BETH contains two binary, manually-labelled flags: `sus` and `evil`. Examples and the explanation of how these labels were created are detailed in Section 2.2. A breakdown of these labels within the subsets for model development is given in Table 4.

Table 4. Breakdown of `sus` and `evil` labels by training, validation, and testing subsets.

DATASET	SUS=0, EVIL=0	SUS=1, EVIL=0	SUS=1, EVIL=1
TRAINING	761875 (99.8%)	1269 (0.02%)	0 (0.00%)
VALIDATION	188181 (99.6%)	786 (0.04%)	0 (0.00%)
TESTING	17508 (9.27%)	13027 (6.89%)	158432 (83.84%)

B. Network Logs

Our BETH dataset is enriched with the availability of corresponding (DNS) network traffic, the frequent starting point when searching for evidence of an intrusion. However, as mentioned, this only covers the activity going into and out of the node, not the events executing within the server. A summary of the network log features is given in Table 5.

C. Testing Dataset Details

This testing dataset was extracted from a single honeypot. The overall attack appears to be instantiating a botnet node. The timeline of the events recorded is provided in Figure 2; this is the typical attack pattern. The server is initially accessed, it may run a few setup operations in the environment to send some details to its Command and Control (C2) for a customised attack, it sleeps for a while, intermittently checks in with the C2 or a clock, and then launches its attack until complete.

In this case, the honeypot is first accessed at around 411 seconds from booting. Several thousand lines are then recorded in the process logs denoting the setup of the new user profile. This happens within milliseconds; these are detailed logs of everything the OS does during the short pause before the terminal opens for user entry when `ssh`-ing into a server. This user then sleeps, pausing all user activity for some number of seconds. This appears to happen at random intervals—a more sophisticated technique than using consistent intervals—of which the latter would give a clear signature of automated activity.

After a few minutes, it sets up an SFTP server to download a file called `dota3.tar.gz` (known botnet malware) and scopes out the system using common commands such as `whoami`, `ls`, and `cat /proc/cpuinfo`. After about 7.5 minutes, it unpacks the `dota3.tar.gz` and runs over a hundred threads, all attempting to connect with different servers.

D. Model & Training Details

Our VAE architecture consists of two 2-layer neural networks with 64 hidden units and ReLU activation functions for the encoder and decoder. The first layer of the encoder concatenates learned embeddings of all input features. The final layer of the decoder outputs a set of logits for categorical distributions for all features. We use a 2D latent representation. Each VAE is trained using the AdamW optimiser (Loshchilov & Hutter, 2018) with learning rate 0.003 and a weight decay of 0.1; early stopping was used with the validation loss. We picked the hidden size $\in \{64, 128, 256\}$, learning rate $\in \{0.003, 0.0003, 0.00003\}$, and weight decay $\in \{0, 0.01, 0.1\}$, using a grid search on the validation loss.

Table 5. The description and type of each feature within the DNS logs.

FEATURE	TYPE	DESCRIPTION
TIMESTAMP	STRING	DATE AND TIME IN THE FORMAT “YYYY-MM-DDTHH:MM:SSZ” FOR WHEN THE PACKET WAS SENT OR RECEIVED
SOURCEIP	STRING	SOURCE IP ADDRESS OF THE PACKET
DESTINATIONIP	STRING	DESTINATION IP ADDRESS OF THE PACKET
DNSQUERY	STRING	THE SENT DNS QUERY (E.G. THE URL SUBMITTED - ”GOOGLE.COM”)
DNSANSWER	LIST OF STRINGS	DNS RESPONSE; CAN BE NULL
DNSANSWERTTL	LIST OF STRINGS (INT)	LIST OF INTEGERS SENT AS STRINGS, CAN BE NULL; THE TIME TO LIVE OF THE DNS ANSWER
DNSQUERYNAMES	LIST OF STRINGS	NAME OF THE REQUESTED RESOURCE
DNSQUERYCLASS	LIST OF STRINGS	CLASS CODE FOR THE RESOURCE QUERY
DNSQUERYTYPE	LIST OF STRINGS	TYPE OF RESOURCE RECORD (A, AAAA, MX, TXT, ETC.)
NUMBEROFANSWERS	STRING (INT)	NUMBER OF ANSWER HEADERS IN THE PACKET
DNSOPCode	STRING (INT)	HEADER INFORMATION REGARDING WHICH OPERATION THIS PACKET WAS SENT (E.G. STANDARD QUERY IS 0)
SENSORID	STRING	SAME AS THE HOSTNAME IN THE PROCESS RECORDS; NAME OF HOST SERVER
SUS	INT (0 OR 1)	BINARY LABEL AS A SUSPICIOUS EVENT (1 IS SUSPICIOUS, 0 IS NOT)
EVIL	INT (0 OR 1)	BINARY AS A KNOWN MALICIOUS EVENT (0 IS BENIGN, 1 IS NOT)

ip-10-100-1-217

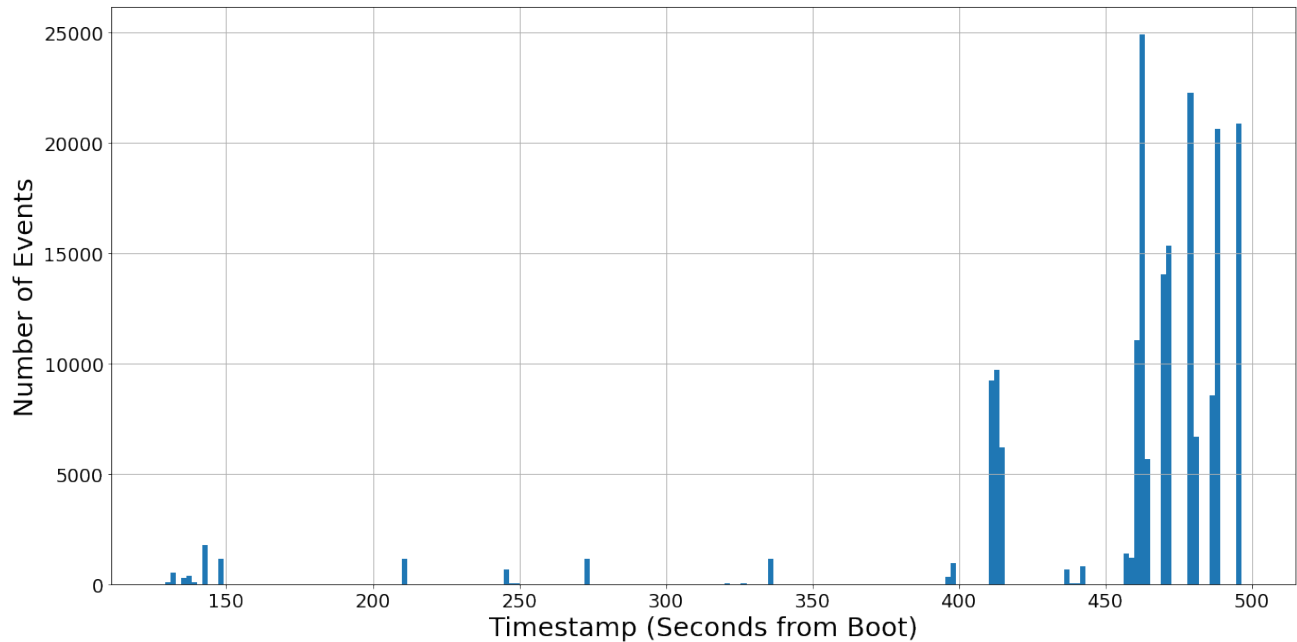


Figure 2. The timeline of the attack captured in the testing dataset is displayed as a histogram based on the number of events and seconds from the “boot” or starting up of the machine.